

Nachum Getzel Elkind

Software Engineer & Product Owner — AI Systems | Cybersecurity
nachumgetzel.elkind@gmail.com · github.com/ngelkind · Beit Shemesh, Israel · 054-595-5698

SUMMARY

I build LLM systems that ship to real customers — and I own them end to end, from the problem to the report a security analyst actually reads. At XM Cyber I am the sole developer and de facto product owner of an AI analysis engine that audits customers' attack-path configurations: I scoped what it should and shouldn't do, defined what counts as a finding worth surfacing, and built the verification and privacy layers that made it safe to put in front of enterprise security teams. I care most about the part of the work where product judgment and engineering meet: deciding what to ship, what to leave out, and how to prove it works. Background in low-level systems and reverse engineering from Israel's Magshimim National Cyber Program.

EXPERIENCE

Solutions Developer — XM Cyber, Tel Aviv | April 2026 – Present

Sole developer and product owner of an LLM-powered configuration-hygiene system for an enterprise attack-path management platform. It audits every attack scenario in a customer environment, finds the ones that are broken, redundant, misnamed, or silently producing nothing, and generates prioritized, environment-specific remediation advice. Also shipped two AI agents into the product. Advisory-only by design; currently migrating from proof of concept into the production codebase.

Product ownership

- **Took an open-ended problem to a shipping product.** Started from a vague complaint — customer attack scenarios silently rot over time — and turned it into a defined product: what counts as a finding, which findings deserve a security team's attention, and what the system deliberately refuses to do.
- **Made the scoping call that made adoption possible.** Chose advisory-and-read-only over auto-remediation, then backed it with a five-layer structural guarantee that the system cannot modify customer data — trading capability for the trust required to run inside a customer environment.
- **Treated prioritization as the core product problem.** Deduplication, root-cause suppression, and confidence-based ranking so a team sees the handful of issues that matter rather than two hundred symptoms of one cause.
- **Designed for the reader, not the engineer.** A report-contract gate that encodes every class of bad output found in manual review as a deterministic pre-publish check, plus a jargon linter so non-technical stakeholders can act on a finding without a translator.
- **Built the measurement, then reported it honestly.** Defined the evaluation before claiming a result: ~88% precision on the high-confidence tier, and a 25-run cross-model comparison that I published as “no statistically significant winner” ($p \approx 0.33$) with the prompt-tuning confound documented, rather than picking the flattering read.
- **Made the product handoff-ready.** A 37-file internal knowledge base and two agent skills so the next engineer can audit why the system judged something the way it did — written for onboarding, not for me.

Engineering

- Architected a four-layer analysis engine (data → comparison → detection → orchestration) with enforced import boundaries and 16 independent detectors — 6 LLM-backed, 10 fully deterministic — backed by ~1,100 automated tests, including false-positive regression suites where every adjudicated false positive is encoded so it can never reappear.
- Built a propose → verify → revise → revert loop that makes LLM-authored advice safe to ship: the model returns structured JSON, nine deterministic verifiers check every claim against platform ground truth, failures get one targeted revision round and then fall back to safe generic text. Every number in the final output is computed in code, never by the model.
- Designed the anonymization layer that makes LLM use viable in a security product: deterministic regex masking with stable hash pseudonyms, a payload contract enforced at both type-check and runtime so no client can skip anonymization, and a per-run deny-list checked at a single chokepoint that aborts on any leak. Shipped an offline audit tool that proves compliance across a full customer environment with zero LLM calls.
- Implemented an entity-inventory query layer and tool-calling loop so the model grounds answers in real data: a lazily-built SQLite index over compressed NDJSON, a boolean expression-tree selector parser, and five read-only tools exposed both in-process and over MCP (JSON-RPC 2.0). Queries the index cannot evaluate return unknown rather than a misleading 0.
- Optimized the comparison hot path with a C extension (bitset Jaccard, popcount intrinsics, size pre-filter), chosen by benchmarking 10 implementations against each other for identical results.

Stack: Python (asyncio, httpx, Pydantic, pytest), C, SQLite, Bash, Git, MCP, LLM APIs

WHAT I BRING

Product

Discovery & scoping — turning an ambiguous problem into a defined product surface, including what to leave out

Prioritization — ranking, deduplication, and root-cause thinking so output respects the user's attention

Metrics & evaluation design — defining success criteria for non-deterministic systems; A/B benchmarking, ground-truth labeling, LLM-as-judge

Writing — specs, internal documentation, and customer-facing output written for the actual audience

Trust & safety tradeoffs — privacy, read-only guarantees, and hallucination containment as product requirements, not afterthoughts

AI / LLM systems — prompt architecture with named, individually-disableable rule blocks for regression bisection; version-gated prompt variants; structured JSON output and schema enforcement; tool-calling loops with bounded rounds; MCP server implementation; hallucination verification and deterministic fallback design; anonymization for model input; provider abstraction across Claude, Gemini, OpenAI, OpenRouter, and internal gateways

Backend & systems — Python asyncio, concurrency control, semaphores, circuit breakers; REST clients with exponential backoff and jitter, token refresh, rate-limit handling, pagination normalization; SQLite indexing, atomic build patterns, schema versioning; C extensions via ctypes with NumPy fallback; client-server and socket programming

Low-level & security — 8086 and AArch64 assembly, OS fundamentals, memory management; reverse engineering and binary analysis (IDA Pro); protocol and packet inspection (Wireshark, Fiddler); least-privilege and read-only architecture design

Languages & tools — Python, C, C++, Assembly, Java, JavaScript, Node.js | Git, pytest, Docker, GitHub Actions, systemd, Wireshark, IDA Pro, Fiddler | Linux, macOS, Windows

PROJECTS

Archon — Multi-Platform AI Assistant — *Python (asyncio), SQLite, systemd, GitHub Actions, Telegram / WhatsApp / Gmail / Google Calendar APIs*

Autonomous agent spanning WhatsApp, Telegram, and Gmail: ingests messages, triages them, and acts through a tool-calling loop — operated entirely from a Telegram bot. ~8,900 lines of Python.

- Built a 78-tool agent catalog behind a scope-filtering registry that hides privileged tools from the model rather than relying on refusal, plus a confirmation gate routing every state-mutating action — send, email, calendar — through owner approval. New chats default to confirm-required.
- Normalized five LLM provider backends — Anthropic content blocks, OpenAI and OpenRouter function-call JSON, Gemini Part objects, and a sandboxed Claude Code subprocess — behind one interface, with a per-call cost model and a hard daily budget gate. Applied prompt caching to the system block to transitively cache all 78 tool schemas.
- Ran eight subsystems as independently supervised asyncio tasks with per-subsystem exponential backoff (5s→300s), so one failing integration cannot take down the others. All durable state in SQLite (WAL), making restarts safe by construction.
- Deployed with zero inbound application ports on a systemd-sandboxed VM (ProtectSystem=strict, NoNewPrivileges), CI that gates on the test suite before deploying, and an egress monitor alerting on any unrecognized outbound host.

Operating System Kernel — *C/C++, AArch64 assembly, QEMU, Docker — in progress*

Kernel written from scratch for 64-bit ARM (AArch64), targeting a working understanding of privilege rings, virtual memory, and the boot path.

- Containerized the cross-compiler toolchain in Docker with QEMU for execution, giving reproducible builds across macOS and Windows and enabling CI checks on every commit.

Client-Server Application — *Python, sockets*

Distributed system over raw sockets with a custom application-level protocol on TCP/IP, concurrent client handling, and structured error handling.

Retail Price Scraper — *Python, web scraping*

Reverse-engineered web requests to work around anti-scraping measures, with data normalization and structured storage for analysis.

EDUCATION

Magshimim National Cyber Program — Israel National Cyber Directorate | 2021 – Present

Elite cyber training program for gifted students; currently in the advanced Magshimim Plus track. Focus on algorithms, low-level programming, networks, and cyber warfare. First-year average 97.5, second-year average 99.7. Reached the national capture-the-flag competition stage.

High School Diploma — Excellence Track | Expected 2027

Overall average 95+. Computer Science (5 units): 100.

LANGUAGES

Russian — native · Hebrew — native · English — fluent, and my day-to-day working language